

The
REAL

ROCK
MARCIANO

HIP HOP INTERACTIVO PARA SUPERCOMUSER

Notas al programa

Rock Marciano es una pieza inspirada en la pieza interactiva de George E. Lewis de nombre Voyager, una pieza que permite un dialogo en tiempo real entre músico y el sistema computarizado asistida por algoritmos generativos y “inteligencia artificial”, en el caso particular de esta pieza se retoma el dialogo en tiempo real con el sistema computarizado y el uso de materiales algorítmicos. Además se inspira en la estética de producciones de Hip Hop y Rap experimental de The Alchemist y Roc Marciano, la pieza conecta las posibilidades interactivas y algorítmicas del Software Supercollider y las posibilidades de workflow de los instrumentos midi nativos del Daw Ableton Live, la pieza permite al interprete por medio de un dispositivo creado con código controlar agrupaciones de instrumentos dentro de Ableton Live que disparados a la par despliegan loops de hip hop, el interprete tiene la oportunidad de controlar estos con sal suma de la dinámica que imprima con su lírica cada determinado tiempo, este puede improvisar o tener una letra escrita.

[illegible]

2. Estructura de Loops:

Los patrones están definidos en archivos externos ("materiales 3.scd" y "patrones 4.scd").

Hay 4 loops principales y una sección de gestos: \loop1 a \loop4: Combinaciones de patrones rítmicos y melódicos, \gestie1: Patrones mas gestuales y libres.

Los patrones usan Pdef (definiciones de patrones) y Ppar/Ptpar (ejecución paralela).

```
(
  // directorio de duraciones para ptpars
  ~durac = 60/90 * 4; // duracion compas 2.66666666666667
  ~durA = ~durac * 34;
  ~durB = ~durac * 32;
  ~durC = ~durac * 2;
  ~durD = ~durac * 5;
  ~durE = ~durac * 6;
  ~durF = ~durac * 16;

  //pdef con sus respectivos Ptpars, esto permite la reproduccion de los ptpars de una manera controlada
  Pdef(\gestie1, Ppar([~ges1, ~ges2, ~ges3, ~ges4]));

  //Pdef(\loop1, Ptpar([0, ~sDrum1, 0, ~instru1a, 0, ~instru1b, 0, ~instru1c, 0, ~instru1d]));
  //Pdef(\loop2, Ptpar([0, ~sDrum2, 0, ~instru2b, 0, ~instru2c, ~durC, ~instru2d, ~durD, ~instru2a]));
  //Pdef(\loop3, Ptpar([0, ~sDrum3, 0, ~instru3c, 0, ~instru3d.finDur(~durE), 0, ~instru3a, ~durC, ~instru3b.finDur(~durE)]));
  //Pdef(\loop4, Ptpar([0, ~sDrum4, ~durD, ~instru4d.finDur(~durF), ~durC, ~instru4a, 0, ~instru4b.finDur(~durF), 3.0, ~instru4c]));

  //tambien se pueben usar como Ppars
  Pdef(\loop1, Ppar([~sDrum1, ~instru1a, ~instru1b, ~instru1c, ~instru1d]));
  Pdef(\loop2, Ppar([~sDrum2, ~instru2a, ~instru2b, ~instru2d, ~instru2c]));
  Pdef(\loop3, Ppar([~sDrum3, ~instru3c, ~instru3d, ~instru3a, ~instru3b]));
  Pdef(\loop4, Ppar([~sDrum4, ~instru4d, ~instru4a, ~instru4b, ~instru4c]));

);
```

3. Detector de Amplitud.

Usa un SynthDef (\dethector) que analiza la amplitud del audio de entrada

Envía valores de amplitud cada 200ms (5Hz) mediante mensajes OSC

Calcula promedios de amplitud en ventanas de 4 y 16 tiempos

Define umbrales para diferentes dinámicas (piano, mezzoforte, forte, fortissimo)

```
////////////////////////////////////////////////////////////////////DETECTOR\DE\AMPLITUDES//////////////////////////////////////////////////////////////////
SynthDef(\dethector, {arg in = 0;
  var inp, ampF, freq, hasFreq;
  inp = SoundIn.ar(in);
  ampF = Amplitude.kr(inp * 100, 0.01, 0.1);
  SendTrig.kr(Impulse.kr(5), 2, ampF);
}).add;

x = Synth(\dethector);

~amps16 = [];
~promamps16 = 0;

~amps4 = [];
~promamps4 = 0;

~ampAct = 0;

~hayamp = false;

~umbralhayamp = 0.001;

// Umbrales de amplitud para cada dinámica
~piano = 0.1;
~mesoforte = 0.3;
~forte = 0.5;
~fortissimo = 0.7;
```

4. Oscdef y Rutinas de control:

Oscdef que convierte las señales del detector en promedios definiendo los mensajes.

Rutina general promedia cada 4 tiempos.

Rutina con casos en los que debe disparar los loops

```
20
27 ///Oscdef convierte las señales del detector en promedios deefiniendo los mensajes
28 OSCdef\deTec, {arg msg, time, addr, port;
29   ~amps16 = ~amps16.add(msg[3]);
30   ~amps4 = ~amps4.add(msg[3]);
31   ~ampAct = msg[3];
32
33   if(msg[3] > ~umbralhayamp)
34   { ~hayamp = true;}
35   {~hayamp = false;};
36
37 }, \tr, s.addr);
38
39 ///rutina general promedia cada 4 tiempos
40 ~rutinaG = r{
41   inf.do{
42     4*4.wait;
43     ~promamps4 = ~amps4.sum/~amps4.size;
44     ~promamps4.postln;
45     ~amps4 = [];}
46 };
47
```

```
129 ~rutine4 = r{
130   inf.do{
131     ~hayamp.postln;
132     ~ampAct.postln;
133     ~promamps16.postln;
134     ~promamps4.postln;
135
136     4.wait(t); // Espera 4 tiempos
137
138     case(
139
140       { ~promamps4 < ~piano }, {
141
142         Pdef(\gestiele1).play;
143         Pdef(\loop1).stop;
144         Pdef(\loop2).stop;
145         Pdef(\loop3).stop;
146         Pdef(\loop4).stop;
147         "gestos".postln
148       },
149
150       { (~promamps4 > ~piano) && (~promamps4 < ~mesoforte)}, {
151
152         Pdef(\gestiele1).stop;
153         Pdef(\loop3).play;
154         Pdef(\loop2).stop;
155         Pdef(\loop1).stop;
156         Pdef(\loop4).stop;
157
158       },
159
160       { (~promamps4 > ~mesoforte) && (~promamps4 < ~forte)}, {
161
162         Pdef(\gestiele1).stop;
163         Pdef(\loop1).stop;
164         Pdef(\loop2).play;
165         Pdef(\loop3).stop;
166         Pdef(\loop4).stop;
167
168       },
169
170       { (~promamps4 > ~forte) && (~promamps4 < ~fortissimo)}, {
171
172         Pdef(\gestiele1).stop;
173         Pdef(\loop1).stop;
174         Pdef(\loop2).stop;
175         Pdef(\loop4).play;
176         Pdef(\loop3).stop;
177
178       },
179
180       { ~promamps4 > ~fortissimo }, {
181
182         Pdef(\gestiele1).stop;
183         Pdef(\loop1).stop;
184         Pdef(\loop2).stop;
185         Pdef(\loop3).stop;
186         Pdef(\loop1).play;
187
188       }
189     )
190   }
191 }
192
```

El disco está casi lleno
Ahorra espacio al optimizar almacenamiento.
Cerrar
Administrar...

Interpreter: Active Server: 0.00% 0.00% 0u 0s 0g 0d 0.0dB M R

5. Interfaz Gráfica

Botones para iniciar/detener el sistema y configurar tempo

Sliders para ajustar los umbrales de amplitud

Visualización del estado actual y nivel de amplitud

Funcionalidad para detener todos los sonidos MIDI

```
//////GUI//////////

//ventana de control
w = Window("TheR.A.", Rect(200, 200, 600, 400)).front;
w.view.decorator = FlowLayout(w.view.bounds);

// Botón de inicio
Button(w, Rect(0,0,280,50))
.states_([["INICIAR PATCH", Color.red(0.7), Color.green(0.7)]])
.action_({
    // Iniciar OSCdef y rutinas

    ~deter = OSCdef(\deTec, {arg msg, time, addr, port;
        ~amps16 = ~amps16.add(msg[3]);
        ~amps4 = ~amps4.add(msg[3]);
        ~ampAct = msg[3];

        ~rutinaG.play(t);

        if(msg[3] > ~umbralhayamp)
        { ~hayamp = true;}
        {~hayamp = false;};...

    }, \tr, s.addr);
    ~monitorRoutine = [~rutine16, ~rutine4].do(_.play(t));
    "Patch iniciado".postln;
});
Button(w, Rect(0,0,280,50))
.states_([["TempoClock", Color.blue(0.7), Color.yellow(0.7)]])
.action_({
    t = TempoClock(60/90);
    "clockclockiado".postln;
});
// Botón de parada
Button(w, Rect(0,0,280,50))
.states_([["DETENER PATCH", Color.white, Color.black]])
.action_({
    ~deter.free;
    ~rutine4.stop;
    ~rutinaG.stop;
    Pdef(\gestiele1).stop;
    Pdef(\loop1).stop;
    Pdef(\loop2).stop;
    Pdef(\loop3).stop;
    Pdef(\loop4).stop;

    4.do{||
        i.postln;
        ~midiOut.allNotesOff(i);
        ~midiOut2.allNotesOff(i);
        ~midiOut3.allNotesOff(i);
        ~midiOut4.allNotesOff(i);
        ~midiOut5.allNotesOff(i);
```

```

EZSlider(w, 280@30, "Umbral hay amp", \amp.asSpec,
  { lez1| ~umbralhayamp = ez1.value },
  ~umbralhayamp ? 0.00
).value= 0.1;

EZSlider(w, 280@30, "Umbral Piano", \amp.asSpec,
  { lez2| ~piano = ez2.value },
  ~piano ? 0.00
).value= 0.2;

EZSlider(w, 280@30, "Umbral mesoforte", \amp.asSpec,
  { lez3| ~mesoforte = ez3.value },
  ~mesoforte ? 0.00
).value= 0.4;

EZSlider(w, 280@30, "Umbral forte", \amp.asSpec,
  { lez4| ~forte = ez4.value },
  ~forte ? 0.00
).value= 0.6;

EZSlider(w, 280@30, "Umbral fortissimo", \amp.asSpec,
  { lez5| ~fortissimo = ez5.value },
  ~fortissimo ? 0.00
).value= 0.8;

// Visualizador de estado
~statusText = StaticText(w, 280@20)
.string_("Estado: Inactivo")
.align_(\center)
//.name_(\estado);
.background_(Color.grey(0.8));

~updateStatus = {
  var status;
  if(~hayamp.not) {
    status = "Sin señal de audio";
  } {
    case(
      { ~promamps4 < ~piano }, { status = "Pianissimo: Gestos" },
      { (~promamps4 > ~piano) && (~promamps4 < ~mesoforte) }, { status = "Piano: Loop 3" },
      { (~promamps4 > ~mesoforte) && (~promamps4 < ~forte) }, { status = "Mezzoforte: Loop 1" },
      { (~promamps4 > ~forte) && (~promamps4 < ~fortissimo) }, { status = "Forte: Loop 2" },
      { ~promamps4 > ~fortissimo }, { status = "Fortissimo: Loop 4" }
    );
  };
  ~statusText.string = "Estado: " + status + " | Amp: " + ~ampAct.round(0.001);
};

~statusRoutine = Routine({
  inf.do {
    ~updateStatus.();
    4.wait(t);
  }
}).play(AppClock);
)

```

```

4.do{1|1
  i.postln;
  ~midiOut.allNotesOff(i);
  ~midiOut2.allNotesOff(i);
  ~midiOut3.allNotesOff(i);
  ~midiOut4.allNotesOff(i);
  ~midiOut5.allNotesOff(i);
  ~midiOut6.allNotesOff(i);
};
"Patch detenido".postln;
});

// Sliders para umbrales
EZSlider(w, 280@30, "Umbral hay amp", \amp.asSpec,
  { lez1| ~umbralhayamp = ez1.value },
  ~umbralhayamp ? 0.00
).value= 0.1;

EZSlider(w, 280@30, "Umbral Piano", \amp.asSpec,
  { lez2| ~piano = ez2.value },
  ~piano ? 0.00
).value= 0.2;

EZSlider(w, 280@30, "Umbral mesoforte", \amp.asSpec,
  { lez3| ~mesoforte = ez3.value },
  ~mesoforte ? 0.00
).value= 0.4;

EZSlider(w, 280@30, "Umbral forte", \amp.asSpec,
  { lez4| ~forte = ez4.value },
  ~forte ? 0.00
).value= 0.6;

EZSlider(w, 280@30, "Umbral fortissimo", \amp.asSpec,
  { lez5| ~fortissimo = ez5.value },
  ~fortissimo ? 0.00
).value= 0.8;

// Visualizador de estado
~statusText = StaticText(w, 280@20)
.string_("Estado: Inactivo")
.align_(\center)
//.name_(\estado);
.background_(Color.grey(0.8));

~updateStatus = {
  var status;
  if(~hayamp.not) {
    status = "Sin señal de audio";
  } {
    case(

```

2- Descripción del archivo scd patrones

Este código define una estructura modular de patrones MIDI (Pbind) organizados en 5 categorías principales, cada una conectada a un bus MIDI diferente para su envío a Ableton Live. El sistema utiliza generación procedural basada en los materiales previamente definidos (grooves, escalas, progresiones armónicas).

Estructura General:

6 buses MIDI (salidas a Ableton via IAC Driver)

5 grupos de instrumentos + 1 sección de gestos

Cada grupo contiene 4 variantes (ej: ~instru1a, ~instru2a, etc.)

Basado en Pbind (patrones deterministas) y Pbindf (variaciones sobre un Pbind base)

```
////////////////////////////////PBINDS//MIDIS//DIRECTORIOS//////////////////////////////////
//ESTOS DIRECTORIOS SE COMPONEN POR UN PBIND MIDI PRINCIPAL Y 3 PBINDF`S QUE ACEN ALUCION A LOS PATRAMETROS PRINCIPALES DEL
PBIND ORIGINAL.

//LO IMPORTANTE DENTRO DE ESTOS SON SU MIDIOUT, SU DEGREE, SU DUR Y SU CHAN. EL MIDIOUT ESTA CONECTADO AL BUS MIDI ASIGNADO A
ABLETON Y CHAN ESPECIFICA EL CANAL AL CUAL SE ASIGNA. EL DEGREE COMO EL DUR TOMA LOS MATERIALES DEFINIDOS EN EL ARCHIVO SCD
MATERIALES.

//

//////////DIRECTORIO//PBINDS//DRUMS//BUS 1
(
~sDrum1 =Pbind(\type, \midi,
  \midiout, ~midiOut,
  \octave, 3,
  \degree, Pseq(~groove1, inf),//~strG[0],
  \dur, ~strD[0],
  \chan, 0,
  \amp, Pseq([0.65, 0.25, 0.25, 0.5, 0.5, 1.0, 0.25, 0.40], inf),
  \rel, 4,
);

~sDrum2 =Pbindf(~sDrum1,
  \degree, Pseq(~groove2, inf),
  \dur, Pseq([~strD[3]],inf),
  \chan, 1,
);

~sDrum3 =Pbindf(~sDrum1,
  \degree, Pseq(~groove3, inf),
  \dur, Pseq([0.25], inf),
  \chan, 2,
);

~sDrum4 =Pbindf(~sDrum1,
  \degree, Pseq(~groove4, inf),
  \dur, Pseq([~strD[3]],inf),
  \chan, 3,
);
```

3.- Descripción materiales.

1. Grooves (Patrones Rítmicos)

Contiene cuatro patrones rítmicos principales (~groove1 a ~groove4), cada uno con variaciones en acentuación y sincopas.

Estructura: Cada groove es un array de 16 elementos (un compás de 4/4 subdividido en semicorcheas).

2. Duraciones (Durs)

Define diferentes patrones de duración para crear variaciones rítmicas:

~durs1 a ~durs7: Arrays con valores de duración en tiempos (ej: 0.25 = semicorchea, Rest(0.5) = silencio de corchea).

Algunos incluyen subdivisiones irregulares (ej: 0.333, 0.65) para grooves más orgánicos.

3. ShannonFingers (Generación Aleatoria Controlada)

~markys y ~markyd:

Usan el algoritmo ShannonFinger (similar a cadenas de Markov) para generar variaciones de los grooves y duraciones.

~strG y ~strD son streams que permiten recorrer los datos de forma no lineal, con parámetros como order (profundidad de memoria) y repeats.

4. Progresiones Armónicas (Acordes y Voicings)

a) Notas de Acorde (Grados)

Definiciones de grados en arrays:

~i a ~vii: Triadas (ej: [0,2,4] = Do mayor).

~todas: Colección de todas las triadas disponibles.

b) Voicings (Disposición de Acordes)

~voicing1 a ~voicing4:

Distribuciones verticales de notas (ej: [3,4,5] = disposición).

Variaciones (~voicing1a, ~voicing1b, etc.) para evitar repeticiones monótonas.

c) Progresiones de Acordes (Chord Progressions)

~prog1 a ~prog4:

Secuencias armónicas basadas en grados (ej: [~ii, ~v, ~i] = II-V-I).

Versiones simplificadas (~prog1a, ~prog1b, etc.) para líneas melódicas.

d) Generación de Streams Armónicos

~strA, ~strB, ~strC, ~strE:

Streams de ShannonFinger aplicados a las progresiones, permitiendo variaciones estocásticas controladas.

Parámetros como order y repeats ajustan la aleatoriedad.

```

(
/////////grooves/////////
~groove1 = [[0, 4b], [0, 4b], 4b, 0, [2, 4b], [0, 4b], 4b, [0, 4b], [0, 4b], 4b, 6b, 0, [2, 6b], [0, 4b], 4b, [1, 6b]];
~groove2 = [[0, 4b], 4b, 4b, [0, 6b], [2, 4b], [0, 6b], 4b, [0, 4b], [0, 4b], 4b, 6b, 4b, [2, 4b], [0, 4b], 4b, [0, 4b]];
~groove3 = [[0, 4b], 4b, [2, 4b], [0, 6b], [2, 4b], [0, 6b], [0, 4b], 4b, [0, 4b], 4b, 6b, 4b, [2, 4b], [0, 4b], 4b, [0, 4b]];
~groove4 = [[0, 6b], 4b, 6b, 4b, [2, 6b], [0, 6b], 4b, [0, 4b], [0, 4b], 4b, 6b, 4b, [2, 4b], [0, 4b], 4b, [0, 4b]];
/////////duraciones/////////

~durs1 = [0.25, 0.25, 0.5, 0.125, 0.75, 0.25, 0.5, 0.25, 0.25, 0.125, 0.75, 0.5, 0.25, 0.125, 0.25, 0.5];
~durs2 = [0.5, 0.25, 0.25, 0.125, Rest(0.125), 0.125, 0.125, 0.5, 0.5, Rest(0.25), 0.25, 0.5, 0.25, 0.25, 0.5, 0.25];
~durs3 = [0.5, Rest(0.25), 0.125, 0.125, Rest(0.125), 0.125, 0.25, 0.5, 0.25, Rest(0.5), 0.25, 0.25, 0.5, 0.25];
~durs4 = [0.75, 0.25, Rest(0.5), 0.125, 0.125, Rest(0.35), 0.65, 0.5, Rest(0.125), 0.125, 0.25, Rest(0.75), 0.35, 1, 0.75, 0.35];
~durs5 = [0.75, 0.25, 0.333, 0.125, 0.5, Rest(0.35), 0.65, Rest(0.125), 0.35];
~durs6 = [ 0.25 , 0.35, 0.45, 0.5, 0.65, 0.75, 0.80, 0.90, 1];
~durs7 = [ 0.25 , 0.5, 1, 1.25, 1.50, 1.75, 2, Rest(0.25), Rest(0.5), Rest(1), Rest(1.25), Rest(1.50), Rest(1.75), Rest(2)];

//ENTRENAR SHANNONFINGERS CON ARRAYS DE ARRAYS
~groovis = [~groove1, ~groove2, ~groove3, ~groove4].flatten;
~dursys = [~durs1, ~durs2, ~durs3, ~durs4, ~durs5, ~durs6, ~durs7].flatten;
~markys = ~groovis.collect{|x| ShannonFinger.new.data =x.asArray};
~markyd = ~dursys.collect{|x| ShannonFinger.new.data =x.asArray};
~strG = ~markys.collect{|x| x.asStream(order: 8, repeats: inf)};
~strD = ~markyd.collect{|x| x.asStream(order: 8, repeats: inf)};

// en tus pibains poner ~strim[0], ~strim[1], etc

~i = [0,2,4];
~ii = [1,3,5];
~iii = [2,4,6];
~iv = [3,5,0];
~v = [4,6,1];
~vi = [5,0,2];
~vim = [5,0b,2];

```

```

~voicing1 = [[3,4,5], [5,4,3], [4,5,0]];
~voicing1a = [3, 5, 4];
~voicing1b = [4, 4, 5];
~voicing1c = [5, 3, 6];
~prog1 = [~ii, ~v, ~i];
~prog1a = [~iia, ~va, ~ia].flatten;
~prog1b = [~iib, ~vb, ~ib].flatten;
~prog1c = [~iic, ~vc, ~ib].flatten;

~loopi1 = [~prog1, ~prog1a, ~prog1b, ~prog1c].flatten;
~looparmoni1 = ~loopi1.collect{|x| ShannonFinger.new.data =x.asArray};
~strA = ~looparmoni1.collect{|x| x.asStream(order: 8, repeats: inf)};

///segundo loop prog, voicing
~voicing2 = [[3,4,5], [5,4,3], [4,5,6], [3,4,5]];
~voicing2a = [3, 5, 4, 3];
~voicing2b = [4, 4, 5, 4];
~voicing2c = [5, 3, 6, 5];
~prog2 = [~iv, ~i, ~v, ~iv];
~prog2a = [~iva, ~ia, ~va, ~iva].flatten;
~prog2b = [~ivb, ~ib, ~vb, ~ivb].flatten;
~prog2c = [~ivc, ~ic, ~vc, ~ivc].flatten;

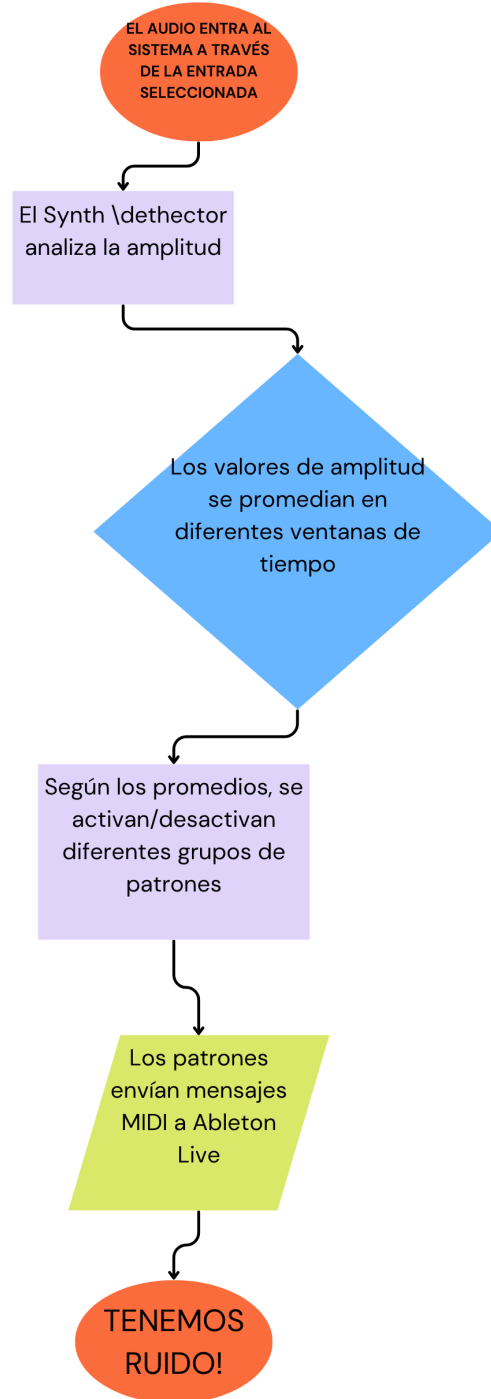
~loopi2 = [~prog2, ~prog2a, ~prog2b, ~prog2c].flatten;
~looparmoni2 = ~loopi2.collect{|x| ShannonFinger.new.data =x.asArray};
~strB = ~looparmoni2.collect{|x| x.asStream(4, 16)};

///tercer loop prog, voicing
~voicing3 = [[3,4,5], [5,4,3], [4,5,6], [5,4,3]];
~voicing3a = [3, 5, 4, 5];
~voicing3b = [4, 4, 5, 4];
~voicing3c = [5, 3, 6, 3];
~prog3 = [~i, ~i, ~v, ~iv];
~prog3a = [~ia, ~ia, ~va, ~iva].flatten;
~prog3b = [~ib, ~ib, ~vb, ~ivb].flatten;
~prog3c = [~ic, ~ic, ~vc, ~ivc].flatten;

~loopi2 = [~prog2, ~prog2a, ~prog2b, ~prog2c].flatten;

```

Funcionamiento del dispositivo



Notas de ejecución

Uso Recomendado

Configurar Ableton para recibir MIDI de los buses IAC 1-6.

Ajustar los umbrales según el instrumento/audio de entrada.

Iniciar el sistema solo evaluando el archivo main.

Controlar los patrones mediante variaciones de amplitud.

Usar la interfaz para monitorear y ajustar en tiempo real.

